

Understanding inference deployment types

As illustrated in *Figure 10.1*, you can choose from three fundamental deployment types when serving models:

- Online real-time inference
- Asynchronous inference
- Offline batch transform

When selecting one design over the other, there is a trade-off between latency, throughput, and costs. You must consider how the data is accessed and the infrastructure you are working with. Another criterion you have to consider is how the user will interact with the model. For example, will the user use it directly, like a chatbot, or will it be hidden within your system, like a classifier that checks if an input (or output) is safe?

You have to consider the freshness of the predictions as well. For example, serving your model in offline batch mode might be easier to implement if, in your use case, it is OK to consume delayed predictions. Otherwise, you have to serve your model in real-time, which is more infrastructure-demanding. Also, you have to consider your application's traffic. Ask yourself questions such as, "Will the application be constantly used, or will there be spikes in traffic and then flatten out?"

With that in mind, let's explore the three major ML deployment types.

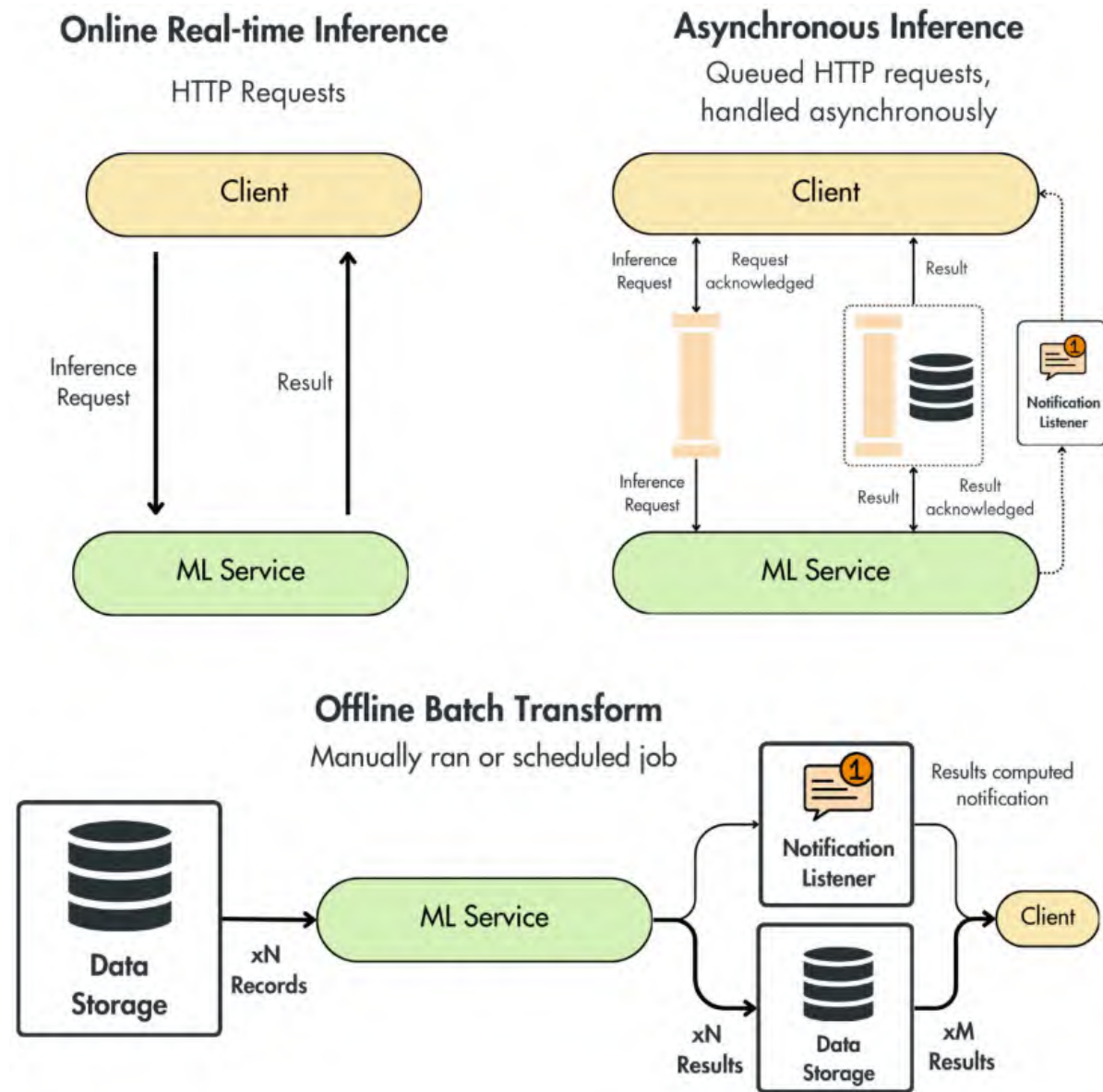


Figure 10.1: The three fundamental architectures of inference deployment types

Online real-time inference

In real-time inference, we have a simple architecture based on a server that can be accessed through HTTP requests. The most popular options are to implement a REST API or gRPC server. The REST API is more accessible but slower, using JSON to pass data between the client and server.

This approach is usually taken when serving models outside your internal network to the broader public. For example, OpenAI's API implements a REST API protocol.

On the other hand, implementing a gRPC makes your ML server faster, though it may reduce its flexibility and general applicability. You have to implement protobuf schemas in your client application, which are more tedious to work with than JSON structures. The benefit, however, is that protobuf objects can be compiled into bites, making the network transfers much faster. Thus, this protocol is often adopted for internal services within the same ML system.

Using the real-time inference approach, the client sends an HTTP request to the ML service, which immediately processes the request and returns the result in the same response. This synchronous interaction means the client waits for the result before moving on.

To make this work efficiently, the infrastructure must support low-latency, highly responsive ML services, often deployed on fast, scalable servers. Load balancing is crucial to evenly distribute incoming traffic evenly, while autoscaling ensures the system can handle varying loads. High availability is also essential to keeping the service operational at all times.

For example, this architecture is often present when interacting with LLMs, as when sending a request to a chatbot or API (powered by LLMs), you expend the predictions right ahead. LLM services, such as ChatGPT or Claude, often use WebSockets to stream each token individually to the end user, making the interaction more responsive. Other famous examples are AI services such as embedding or reranking models used for **retrieval-augmented generation (RAG)** or online recommendation engines in platforms like TikTok.

The simplicity of real-time inference, with its direct client-server interaction, makes it an attractive option for applications that require immediate responses, like chatbots or real-time recommendations. However, this approach can be challenging to scale and may lead to underutilized resources during low-traffic periods.

Asynchronous inference

In asynchronous inference, the client sends a request to the ML service, which acknowledges the request and places it in a queue for processing. Unlike real-time inference, the client doesn't wait for an immediate response. Instead, the ML service processes the request asynchronously. This requires a robust infrastructure that queues the messages to be processed by the ML service later on.

When the results are ready, you can leverage multiple techniques to send them to the client. For example, depending on the size of the result, you can put it either in a different queue or an object storage dedicated to storing the results.

The client can either adopt a polling mechanism that checks on a schedule if there are new results or adopt a push strategy and implement a notification system to inform the client when the results are ready.

Asynchronous inference uses resources more efficiently. It doesn't have to process all the requests simultaneously but can define a maximum number of machines that run in parallel to process the messages. This is possible because the requests are stored in the queue until a machine can process them. Another huge benefit is that it can handle spikes in requests without any timeouts. For example, let's assume that on an e-shop site, we usually have 10 requests per second handled by two machines. Because of a promotion, many people started to visit the site, and the number of requests spiked to 100 requests per second. Instead of scaling the number of **virtual machines (VMs)** by 10, which can add drastic costs, the requests are queued, and the same two VMs can process them in their rhythm without any failures.

Another popular advantage for asynchronous architectures is when the requested job takes significant time to complete. For example, if the job takes over five minutes, you don't want to block the client waiting for a response.

While asynchronous inference offers significant benefits, it does come with trade-offs. It introduces higher latency, making it less suitable for time-sensitive applications. Additionally, it adds complexity to the implementation and infrastructure. Depending on your design choices, this architecture type falls somewhere between online and offline, offering a balance of benefits and trade-offs.

For example, this is a robust design where you don't care too much about the latency of the inference but want to optimize costs heavily. Thus, it is a popular choice for problems such as extracting keywords from documents, summarizing them using LLMs, or running deep-fake models on top of videos. But suppose you carefully design the autoscaling system to process the requests from the queue at decent speeds. In that case, you can leverage this design for other use cases, such as online recommendations for e-commerce. In the end, it sums up how much computing power you are willing to pay to meet the expectations of your application.

Offline batch transform

Batch transform is about processing large volumes of data simultaneously, either on a schedule or triggered manually. In a batch transform architecture, the ML service pulls data from a storage system, processes it in a single operation, and then stores the results in storage. The storage system can be implemented as an object storage like AWS S3 or a data warehouse like GCP BigQuery.

Unlike the asynchronous inference architecture, a batch transform design is optimized for high throughput with permissive latency requirements. When real-time predictions are unnecessary, this approach can significantly reduce costs, as processing data in big batches is the most economical method. Moreover, the batch transform architecture is the simplest way to serve a model, accelerating development time.

The client pulls the results directly from data storage, decoupling its interaction with the ML service. Taking this approach, the client never has to wait for the ML service to process its input, but at the same time, it doesn't have the flexibility to ask for new results at any time. You can see the data storage, where the results are stored as a large cache, from where the client can take what is required. If you want to make your application more responsive, the client can be notified when the processing is complete and can retrieve the results.

Unfortunately, this approach will always introduce a delay between the time the predictions were computed and consumed. That's why not all applications can leverage this design choice. For example, if we implement a recommender system for a video streaming application, having a delay of one day for the predicted movies and TV shows might work because you don't consume these products at a high frequency. But suppose you make a recommender system for a social media platform. In that case, delaying one day or even one hour is unacceptable, as you constantly want to provide fresh content to the user.

Batch transform shines in scenarios where high throughput is needed, like data analytics or periodic reporting. However, it's unsuitable for real-time applications due to its high latency and requires careful planning and scheduling to manage large datasets effectively. That's why it is an offline serving method.

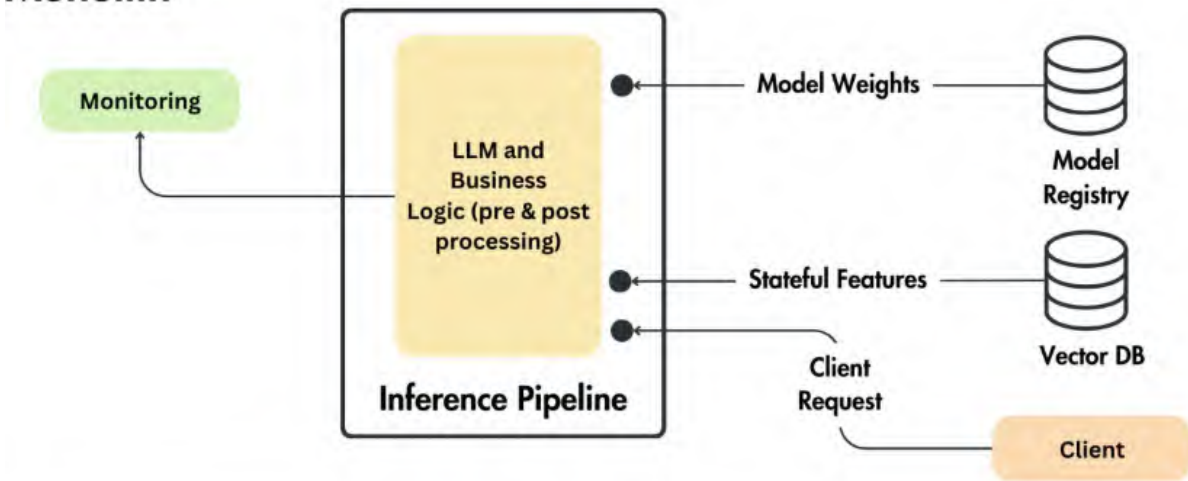
To conclude, we examined the three most common architectures for serving ML models. We started with online real-time inference, which serves clients when they request a prediction. Then, we looked at the asynchronous inference method, which sits between online and offline. Ultimately, we presented the offline batch transform, which is used to process large amounts of data and store them in data storage, from where the client later consumes them.

Monolithic versus microservices architecture in model serving

In the previous section, we saw three different methods of deploying the ML service. The differences in architecture were mainly based on the interaction between the client and the ML service, such as the communication protocol, the ML service responsiveness, and prediction freshness.

But another aspect to consider is the architecture of the ML service itself, which can be implemented as a monolithic server or as multiple microservices. This will impact how the ML service is implemented, maintained, and scaled. Let's explore the two options.

Monolith



Microservices

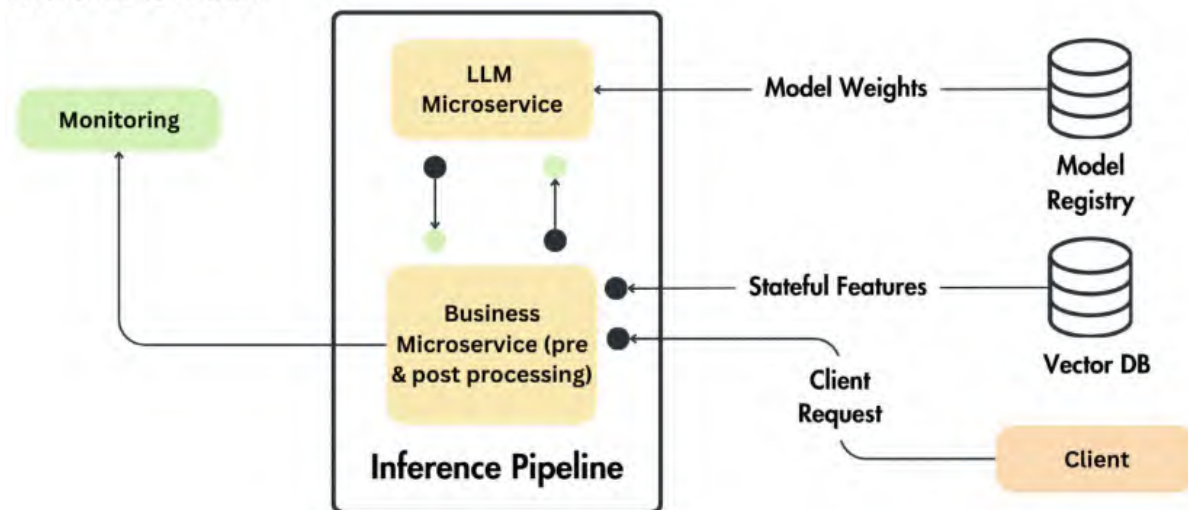


Figure 10.2: Monolithic versus microservices architecture in model serving

Monolithic architecture

The LLM (or any other ML model) and the associated business logic (preprocessing and post-processing steps) are bundled into a single service in a monolithic architecture. This approach is straightforward to implement at the beginning of a project, as everything is placed within one code base. Simplicity makes maintenance easy when working on small to medium projects, as updates and changes can be made within a unified system.

One key challenge of a monolithic architecture is the difficulty of scaling components independently. The LLM typically requires GPU power, while the rest of the business logic is CPU and I/O-bound. As a result, the infrastructure must be optimized for both GPU and CPU. This can lead to inefficient resource use, with the GPU being idle when the business logic is executed and vice versa. Such inefficiency can result in additional costs that could be avoided.

Moreover, this architecture can limit flexibility, as all components must share the same tech stack and runtime environment. For example, you might want to run the LLM using Rust or C++ or compile it with ONNX or TensorRT while keeping the business logic in Python. Having all the code in one system makes this differentiation difficult. Finally, splitting the work across different teams is complex, often leading to bottlenecks and reduced agility.

Microservices architecture

A microservices architecture breaks down the inference pipeline into separate, independent services—typically splitting the LLM service and the business logic into distinct components. These services communicate over a network using protocols such as REST or gRPC.

As illustrated in *Figure 10.3*, the main advantage of this approach is the ability to scale each component independently. For instance, since the LLM service might require more GPU resources than the business logic, it can be scaled horizontally without impacting the other components. This optimizes resource usage and reduces costs, as different types of machines (e.g., GPU versus CPU) can be used according to each service's needs.

For example, let's assume that the LLM inference takes longer, so you will need more ML service replicas to meet the demand. But remember that GPU VMs are expensive. By decoupling the two components, you will run only what is required on the GPU machine and not block the GPU VM with other computing that can be done on a much cheaper machine.

Thus, by decoupling the components, you can scale horizontally as required, with minimal costs, providing a cost-effective solution to your system's needs.

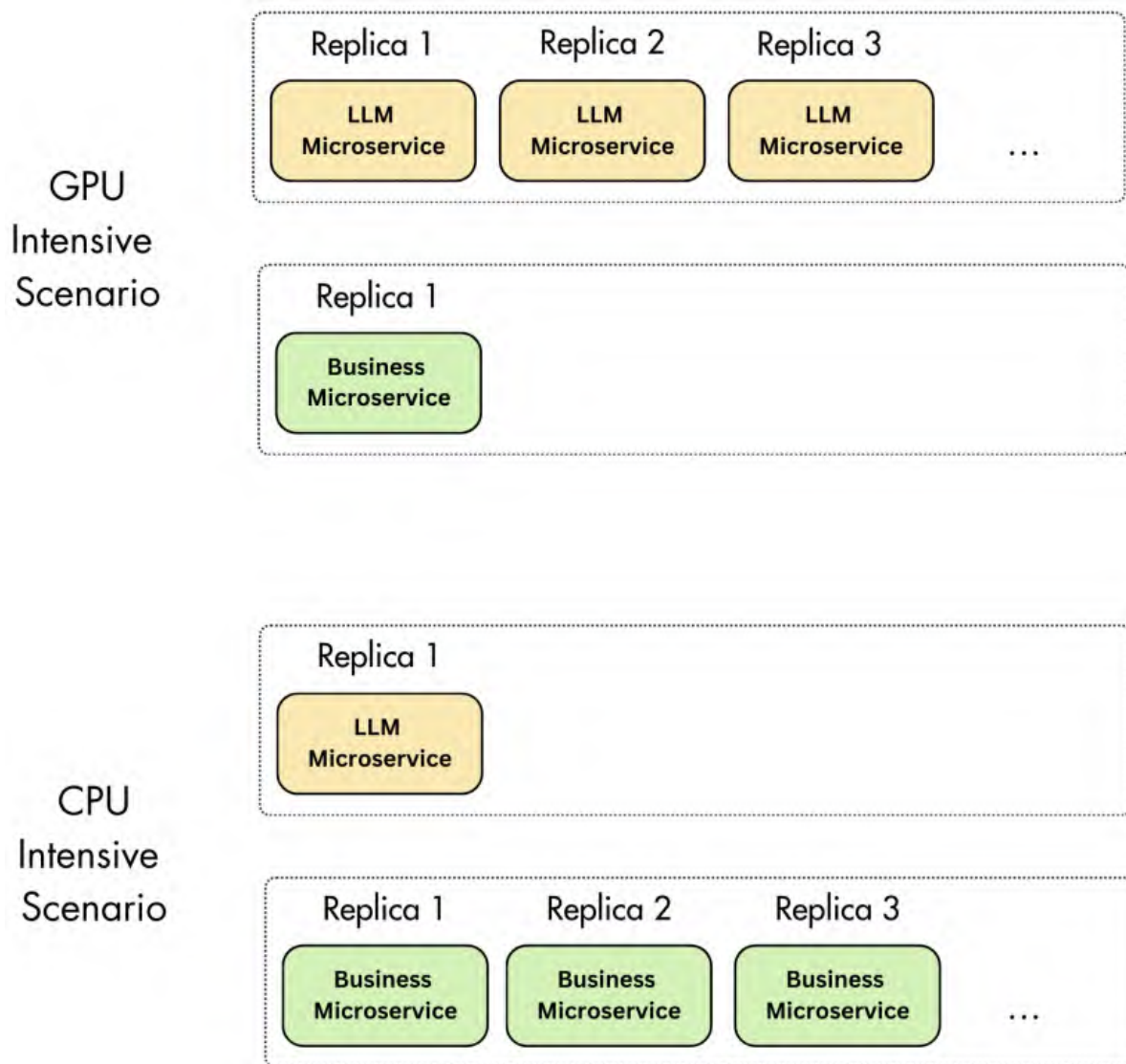


Figure 10.3: Scaling microservices independently based on compute requirements

Additionally, each microservice can adopt the most suitable technology stack, allowing teams to innovate and optimize independently.

However, microservices introduce complexity in deployment and maintenance. Each service must be deployed, monitored, and maintained separately, which can be more challenging than managing a monolithic system.

The increased network communication between services can also introduce latency and potential points of failure, necessitating robust monitoring and resilience mechanisms.

Note that the proposed design for decoupling the ML model and business logic into two services can be extended if necessary. For example, you can have one service for preprocessing the data, one for the model, and another for post-processing the data. Depending on the four pillars (latency, throughput, data, and infrastructure), you can get creative and design the most optimal architecture for your application needs.

Choosing between monolithic and microservices architectures

The choice between monolithic and microservices architectures for serving ML models largely depends on the application's specific needs. A monolithic approach might be ideal for smaller teams or more straightforward applications where ease of development and maintenance is a priority. It's also a good starting point for projects without frequent scaling requirements. Also, if the ML models are smaller, don't require a GPU, or don't require smaller and cheaper GPUs, the trade-off between reducing costs and complicating your infrastructure is worth considering.

On the other hand, microservices, with their adaptability and scalability, are well suited for larger, more complex systems where different components have varying scaling needs or require distinct tech stacks. This architecture is particularly advantageous when scaling specific system parts, such as GPU-intensive LLM services. As LLMs require powerful machines with GPUs, such as Nvidia A100, V100, or A10g, which are incredibly costly, microservices offer the flexibility to optimize the system for keeping these machines busy all the time or quickly scaling down when the GPU is idle. However, this flexibility comes at the cost of increased complexity in both development and operations.

A common strategy is to start with a monolithic design and further decouple it into multiple services as the project grows. However, to successfully do so without making the transition too complex and costly, you must design the monolith application with this in mind. For instance, even if all the code runs on a single machine, you can completely decouple the modules of the application at the software level. This makes it easier to move these modules to different microservices when the time comes. When working with Python, for example, you can implement the ML and business logic into two different Python modules that don't interact with each other. Then, you can glue these two modules at a higher level, such as through a service class, or directly into the framework you use to expose your application over the internet, such as FastAPI.